Group project - Wine - Jared Martin - R Code

Jared Martin

2024-11-24

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(class)
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(e1071)
library(nnet)
library(plotly)
```

```
##
## Attaching package: 'plotly'
##
## The following object is masked from 'package:ggplot2':
##
##     last_plot
```

```
##
## The following object is masked from 'package:stats':
##
##     filter
##
## The following object is masked from 'package:graphics':
##
##     layout
```

```
library(knitr)
```

Group Members: 1 of 1 Jared Martin

Cleaning and Preparing the data

The following code pulls in the two wine data sets, creates a new variable called wine type (red or white), makes quality and type factor variables, joins the two data sets together and saves the data as a csv.

```
rbind(read.csv("./Data/winequality-red.csv", sep = ";") %>%
        mutate(type = "red"),
      read.csv("./Data/winequality-white.csv", sep = ";")%>%
        mutate(type = "white")
                      )  %>%
  write.csv( "./Data/wine_data.csv")

wine_data <- read_csv("./Data/wine_data.csv")
```

```
## New names:
## Rows: 6497 Columns: 14
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): type dbl (13): ...1, fixed.acidity, volatile.acidity, citric.acid,
## residual.sugar...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```
wine_data$quality <- factor(wine_data$quality)

wine_data[,2:14] -> wine_data

wine_data %>%
  dplyr::select(quality, everything()) -> wine_data
```

Research Question #1: Can we predict the quality of a wine given its characteristic?

Each wine was given as ranking for quality between 0-10. Although in the data set the lowest level of quality is 3, while the highest is an 8. Given that there is more then two level of quality we will be using KKN and classification trees to predict the quality of the wine given it's characteristics.

```r
levels(wine_data$quality)
```

```
## [1] "3" "4" "5" "6" "7" "8" "9"
```

Method #1: KNN

1. Spliting the data into a testing and training dataset.

```r
set.seed(1)

n <- nrow(wine_data)

z <- sample(n, n/2)

wine_test <- wine_data[z,]

wine_train <- wine_data[-z,][1:(n/2),]

x.train <- wine_train[,2:12]
x.test <- wine_test[,2:12]

y.train <- wine_train$quality
y.test <- wine_test$quality

dim(x.train)[1] == length(y.train)  # Test to see if length is the same. Should be TRUE
```

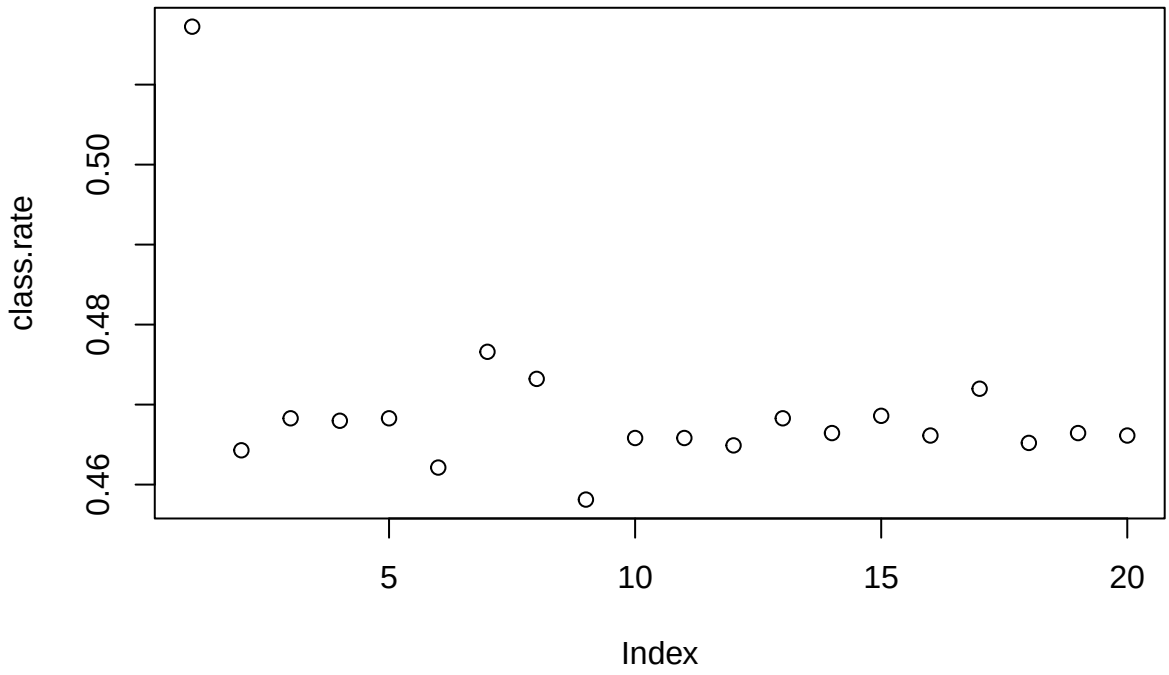
```
## [1] TRUE
```

```r
dim(x.test)[1] == length(y.test)    # Test to see if length is the same. Should be TRUE
```

```
## [1] TRUE
```

2. Determining the optimal number of nearest neighbors "K"

```r
class.rate = rep(0,20)

for (K in 1:20) {
  knn.result = knn(x.train, x.test, y.train, K)
  class.rate[K] = mean( y.test == knn.result )
}

plot(class.rate)
```




```
max(class.rate) # optimal number of K neighbors is 1
```

```
## [1] 0.5172414
```

3. Cross Validation - CClasification Rate and Confusion table using Optimal number of nearest Neighbors.

```
knn_cv_wine_quality <- knn(x.train, x.test, y.train, 1)

table(y.test, knn_cv_wine_quality)
```

```
##       knn_cv_wine_quality
## y.test   3   4   5   6   7   8   9
##      3   0   2   7   2   3   0   0
##      4   4  12  35  41   8   1   0
##      5   5  29 557 373  92   9   0
##      6   2  40 321 844 173  40   0
##      7   1   3  65 214 238  26   1
##      8   0   5   8  30  26  29   0
##      9   0   0   0   1   1   0   0
```

```
knn_cv_classification_rate <- mean(y.test == knn_cv_wine_quality)

mad_knn <- mean(abs(as.numeric(wine_test$quality) - as.numeric(knn_cv_wine_quality)))
```

Method #2: Classification Trees

1. Classification Tree for Quality

```
tree_wine_quality <- tree(quality ~ ., data = wine_train)
```

```
## Warning in tree(quality ~ ., data = wine_train): NAs introduced by coercion
```

```
tree_wine_quality
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 3248 8303.0 6 ( 0.0049261 0.0354064 0.3303571 0.4356527 0.1634852 0.0292488 0.0009236 )
##    2) alcohol < 10.625 1947 4304.0 5 ( 0.0061633 0.0405752 0.4694402 0.4083205 0.0662558 0.0092450 0
##      4) volatile.acidity < 0.2625 799 1812.0 6 ( 0.0050063 0.0225282 0.2941176 0.5394243 0.1201502 0
##      5) volatile.acidity > 0.2625 1148 2257.0 5 ( 0.0069686 0.0531359 0.5914634 0.3170732 0.0287456 0
##       10) alcohol < 9.85 782 1286.0 5 ( 0.0051151 0.0460358 0.6764706 0.2647059 0.0076726 0.0000000 0
##       11) alcohol > 9.85 366  873.3 6 ( 0.0109290 0.0683060 0.4098361 0.4289617 0.0737705 0.0081967 0
##    3) alcohol > 10.625 1301 3308.0 6 ( 0.0030746 0.0276710 0.1222137 0.4765565 0.3089931 0.0591852 0
##      6) alcohol < 11.775 728 1834.0 6 ( 0.0027473 0.0384615 0.1771978 0.5027473 0.2417582 0.0370879 0
##      7) alcohol > 11.775 573 1377.0 6 ( 0.0034904 0.0139616 0.0523560 0.4432810 0.3944154 0.0872600 0
```

```
summary(tree_wine_quality)
```

```
##
## Classification tree:
## tree(formula = quality ~ ., data = wine_train)
## Variables actually used in tree construction:
## [1] "alcohol"          "volatile.acidity"
## Number of terminal nodes:  5
## Residual mean deviance:  2.215 = 7182 / 3243
## Misclassification error rate: 0.4652 = 1511 / 3248
```

```
# not all class are represented as terminal node classifications (3, 4 & 8 are not represented).
```

2. Calculating the Classification Rate for Unprunned Tree

```
tree_hat <- predict(tree_wine_quality, wine_test, type = "class")
```

```
## Warning in pred1.tree(object, tree.matrix(newdata)): NAs introduced by coercion
```

```
table(tree_hat, wine_test$quality)
```

```
##
## tree_hat    3    4    5    6    7    8    9
##        3    0    0    0    0    0    0    0
```

```
##         4    0    0    0    0    0    0    0
##         5    3   43  483  234   11    1    0
##         6   11   58  582 1186  537   97    2
##         7    0    0    0    0    0    0    0
##         8    0    0    0    0    0    0    0
##         9    0    0    0    0    0    0    0
```

```r
tree_classification_rate <- mean(tree_hat == wine_test$quality)
```

3. Prunning the Classification tree on misclassifaction rate.

```r
# The optimal number of terminal nods on a tree is 8. The same number of terminal nods as the tree abov

set.seed(1)

tree_cv_wine_quality <- cv.tree( tree_wine_quality, FUN = prune.misclass )
```

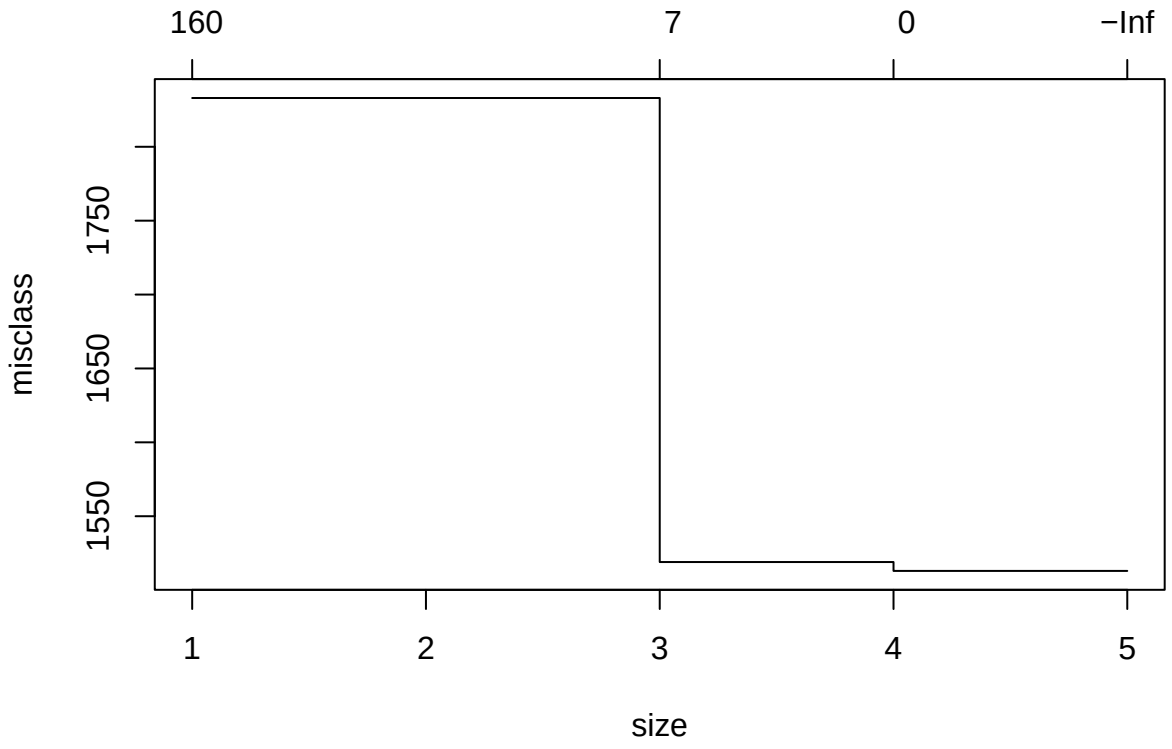
```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
tree_cv_wine_quality
```

```
## $size
## [1] 5 4 3 1
##
## $dev
## [1] 1513 1513 1519 1833
##
## $k
## [1]  -Inf   0.0   7.0 157.5
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

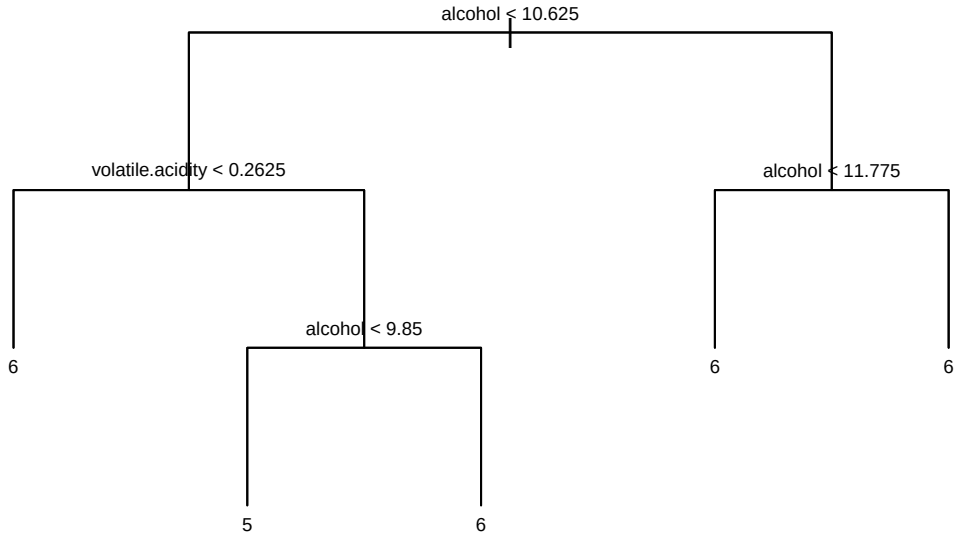
```
plot(tree_cv_wine_quality)
```




```
tree_pruned_wine_quality <- prune.misclass(tree_wine_quality, best = 5)
```

4. Plotting Classification Tree

```
plot(tree_pruned_wine_quality, type = "uniform")
text(tree_pruned_wine_quality, cex = 0.6)
```



5. Cross Validation of the Classification Tree

```r
tree_hat <- predict(tree_pruned_wine_quality, wine_test, type = "class")
```

```
## Warning in pred1.tree(object, tree.matrix(newdata)): NAs introduced by coercion
```

```r
table(tree_hat, wine_test$quality)
```

```
##
## tree_hat    3    4    5    6    7    8    9
##        3    0    0    0    0    0    0    0
##        4    0    0    0    0    0    0    0
##        5    3   43  483  234   11    1    0
##        6   11   58  582 1186  537   97    2
##        7    0    0    0    0    0    0    0
##        8    0    0    0    0    0    0    0
##        9    0    0    0    0    0    0    0
```

```r
tree_pruned_classification_rate <- mean(tree_hat == wine_test$quality)

mad_tree <- mean(abs(as.numeric(wine_test$quality) - as.numeric(tree_hat)))
```

##Method #3: Random Forest

1. Creaing the Traing Random Forest

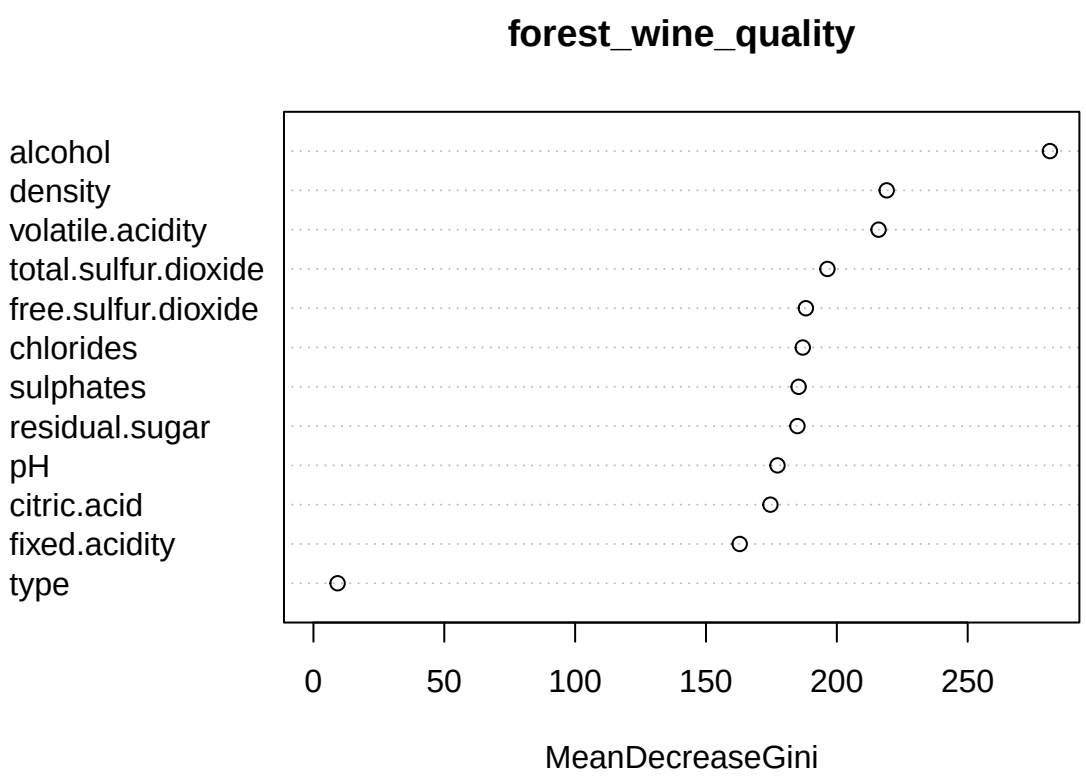
```r
set.seed(1)

forest_wine_quality <- randomForest(quality ~ ., data = wine_train)

forest_wine_quality
```

```
##
## Call:
##  randomForest(formula = quality ~ ., data = wine_train)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 35.04%
## Confusion matrix:
##   3 4   5    6   7  8 9 class.error
## 3 0 2   9    5   0  0 0   1.0000000
## 4 1 8  55   50   1  0 0   0.9304348
## 5 0 0 732  334   7  0 0   0.3178006
## 6 0 3 235 1080  96  1 0   0.2367491
## 7 0 0  19  240 269  3 0   0.4934087
## 8 0 0   1   40  33 21 0   0.7789474
## 9 0 0   0    1   2  0 0   1.0000000
```

```r
importance(forest_wine_quality)
```

```
##                       MeanDecreaseGini
## fixed.acidity               162.881103
## volatile.acidity            215.943020
## citric.acid                 174.611373
## residual.sugar              184.942953
## chlorides                   186.973404
## free.sulfur.dioxide         188.164176
## total.sulfur.dioxide        196.405281
## density                     219.058017
## pH                          177.318223
## sulphates                   185.396477
## alcohol                     281.417878
## type                          9.240769
```

```r
varImpPlot(forest_wine_quality)
```



2.

Searching for the optimal number of trees and number of variables considerd at each branch.

```r
set.seed(1)

err.rate <- numeric(10)  # To store error rates for each mtry
n.trees <- numeric(10)   # To store the number of optimal trees for each mtry

for (m in 1:10) {
  forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                        mtry = m )
  opt.tree <- which.min(forest_wine_quality_m$err.rate)
```

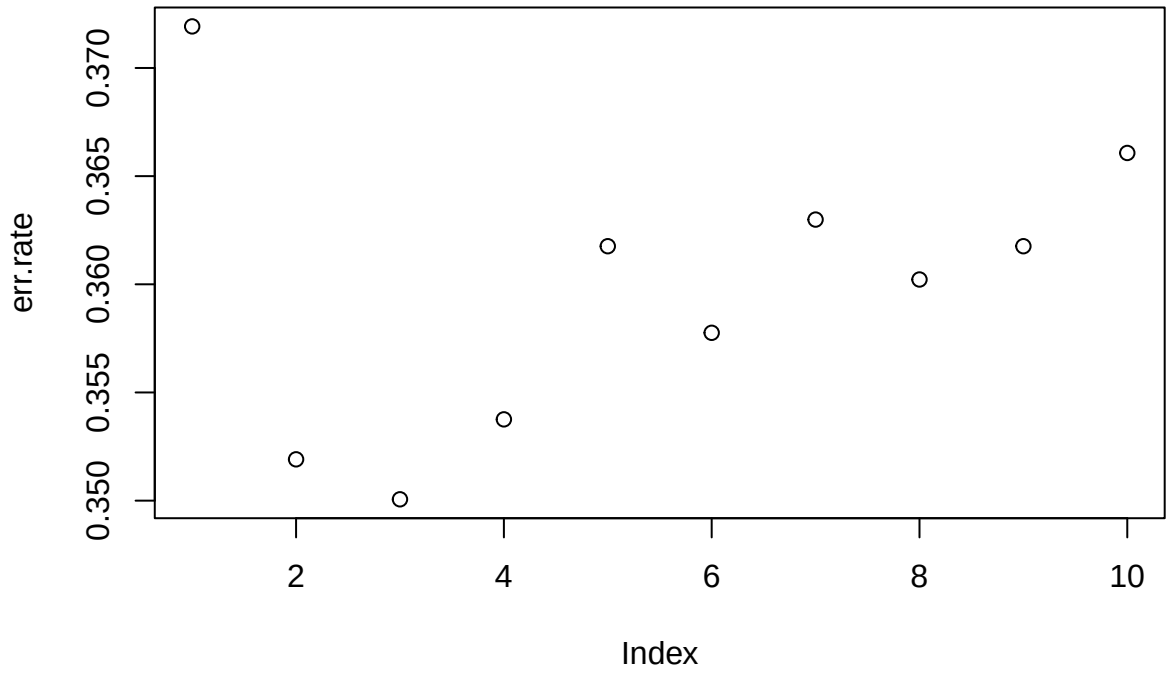
```
  forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                        mtry = m, ntree = opt.tree)
  yhat_rf_m <- predict(forest_wine_quality_m, newdata = wine_test, type = "class")
  err.rate[m] <- 1-mean(wine_test$quality == yhat_rf_m)
  n.trees[m] <- opt.tree
}

err.rate
```

```
##  [1] 0.3719212 0.3519089 0.3500616 0.3537562 0.3617611 0.3577586 0.3629926
##  [8] 0.3602217 0.3617611 0.3660714
```

```
n.trees
```

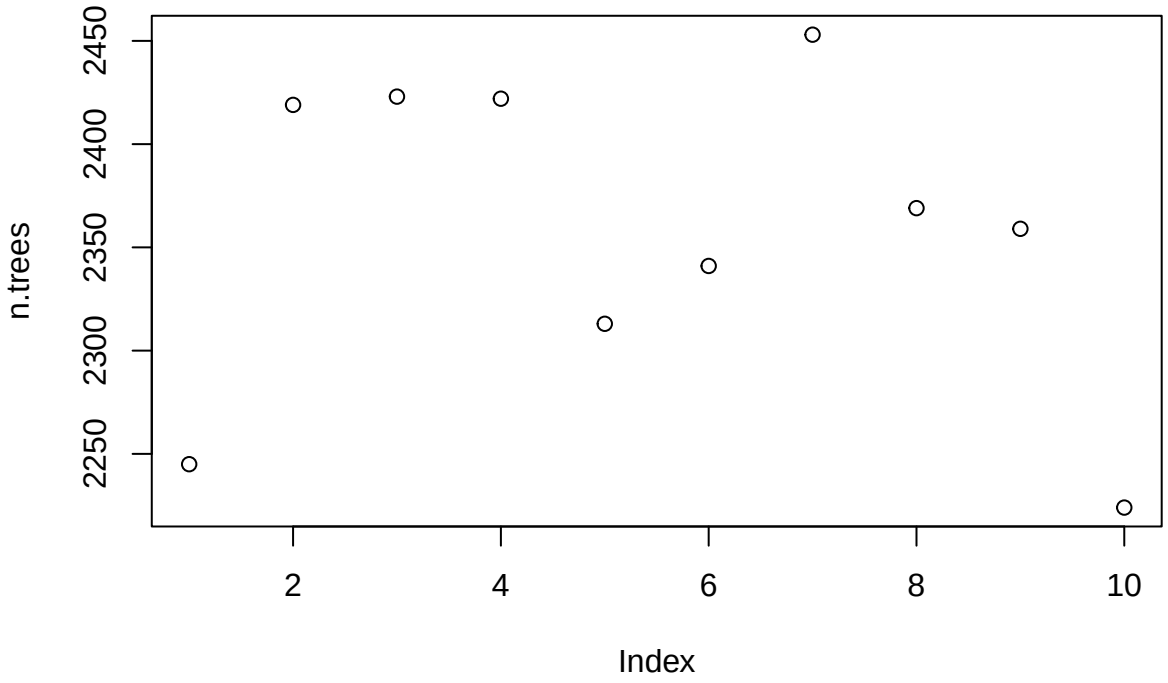
```
##  [1] 2245 2419 2423 2422 2313 2341 2453 2369 2359 2224
```

```
plot(err.rate); line(err.rate)
```




```
##
## Call:
## line(err.rate)
##
## Coefficients:
## [1]  0.348471  0.001591
```

```
plot(n.trees); line(n.trees)
```




```
##
## Call:
## line(n.trees)
##
## Coefficients:
## [1]  2440.792    -9.417
```

```
# optimal solution is 2192 trees and 2 variables considered at each branch.
```

3. Cross Validation with Optimal number of trees and number of variables considerd at each branch.

```
set.seed(1)

forest_cv_wine_quality <- randomForest(quality ~ ., data = wine_train,
                                       mtry = 1, ntree = 2480 )

yhat_rf <- predict(forest_cv_wine_quality, newdata = wine_test, type = "class")

forest_classification_rate <- mean(wine_test$quality == yhat_rf)

mad_forest <- mean(abs(as.numeric(wine_test$quality) - as.numeric(yhat_rf)))
```

Method #4: Support Vecot Machines

1. Creating a SVM model with a linear bounder

```r
SVM_wine_quality <- svm(quality ~ ., data = wine_train, kernel="linear")

summary(SVM_wine_quality)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "linear")
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  1
##
## Number of Support Vectors:  3017
##
##  ( 915 531 115 1342 95 16 3 )
##
##
## Number of Classes:  7
##
## Levels:
##  3 4 5 6 7 8 9
```

2. Tuning the SVM model to detiermen the optimal kernel and cost.

```r
set.seed(1)

svm_tuned <- tune(
  svm,
  quality ~ .,
  data = wine_train,
  ranges = list(
    cost = 10^seq(-2, 2, by = 1),
    kernel = c("linear", "polynomial", "radial", "sigmoid")  # Test multiple kernels
  )
)
```

```r
summary(svm_tuned)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost kernel
##    10 radial
##
## - best performance: 0.4220921
```

```
##
## - Detailed performance results:
##     cost     kernel     error dispersion
## 1  1e-02     linear 0.4682887 0.01900000
## 2  1e-01     linear 0.4642830 0.02078634
## 3  1e+00     linear 0.4627455 0.02002080
## 4  1e+01     linear 0.4636686 0.01985583
## 5  1e+02     linear 0.4633609 0.01938088
## 6  1e-02 polynomial 0.5584834 0.02513259
## 7  1e-01 polynomial 0.5190826 0.02450784
## 8  1e+00 polynomial 0.4565717 0.02599846
## 9  1e+01 polynomial 0.4390209 0.03113474
## 10 1e+02 polynomial 0.4528851 0.02800888
## 11 1e-02     radial 0.5643324 0.02489330
## 12 1e-01     radial 0.4673542 0.02150864
## 13 1e+00     radial 0.4297930 0.02438804
## 14 1e+01     radial 0.4220921 0.03318718
## 15 1e+02     radial 0.4270237 0.02437081
## 16 1e-02    sigmoid 0.5341605 0.03003334
## 17 1e-01    sigmoid 0.4772137 0.02291533
## 18 1e+00    sigmoid 0.5584976 0.02326672
## 19 1e+01    sigmoid 0.5652887 0.03971605
## 20 1e+02    sigmoid 0.5757540 0.02641117
```

```
# optimal cost is 10 with a radial boundary.
```

3. Cross Validation with Optimal Cost and Kernal

```
SVM_wine_quality_optimal <- svm(quality ~ .,
                                data = wine_train,
                                kernel="radial",
                                cost = 10)

summary(SVM_wine_quality_optimal)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "radial",
##     cost = 10)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  2793
##
##  ( 843 503 114 1219 95 16 3 )
##
##
```

```
## Number of Classes:  7
##
## Levels:
##  3 4 5 6 7 8 9
```

```
svm_yhat <- predict(SVM_wine_quality_optimal, data = wine_test)

length(svm_yhat)
```

```
## [1] 3248
```

```
length(wine_test$quality)
```

```
## [1] 3248
```

```
table(svm_yhat, wine_test$quality)
```

```
##
## svm_yhat   3   4   5   6   7   8   9
##        3   0   0   1   7   1   0   0
##        4   0   1  12  17   6   4   0
##        5   5  34 352 469 167  27   2
##        6   7  53 602 788 316  58   0
##        7   2  13  95 130  55   9   0
##        8   0   0   3   9   3   0   0
##        9   0   0   0   0   0   0   0
```

```
svm_cv_classification_rate <- mean(svm_yhat == wine_test$quality)

mad_svm <- mean(abs(as.numeric(wine_test$quality) - as.numeric(svm_yhat)))
```

Method #5: Artifical Neural Nextwork

1. Searching for the optimal size of hiden nodes

```
set.seed(1)

err.rate <- numeric(10)

for (m in 1:12) {
  ann_wine_quality <- nnet(quality ~ .,
                           data = wine_train,
                           size = m)
  yhat_ann <- predict(ann_wine_quality, newdata = wine_test, type = "class")
  err.rate[m] <- 1-mean(wine_test$quality == yhat_ann)
}
```

```
## # weights:  27
## initial  value 5212.598328
```

```
## iter  10 value 4150.169951
## iter  20 value 4120.599867
## iter  30 value 4086.774995
## iter  40 value 3870.792464
## iter  50 value 3610.285149
## iter  60 value 3553.767130
## iter  70 value 3538.673368
## iter  80 value 3533.191430
## iter  90 value 3532.014817
## iter 100 value 3529.961545
## final  value 3529.961545
## stopped after 100 iterations
## # weights:  47
## initial  value 6170.810951
## iter  10 value 4216.777275
## iter  20 value 4151.547248
## iter  30 value 4144.425982
## iter  40 value 4129.732266
## iter  50 value 4114.689192
## iter  60 value 4022.250327
## iter  70 value 3922.511207
## iter  80 value 3705.584964
## iter  90 value 3615.116206
## iter 100 value 3556.039910
## final  value 3556.039910
## stopped after 100 iterations
## # weights:  67
## initial  value 7919.988519
## iter  10 value 4281.127946
## iter  20 value 4133.682121
## iter  30 value 4115.764681
## iter  40 value 4031.892416
## iter  50 value 3795.157538
## iter  60 value 3630.799628
## iter  70 value 3568.732963
## iter  80 value 3541.507030
## iter  90 value 3532.256106
## iter 100 value 3530.109504
## final  value 3530.109504
## stopped after 100 iterations
## # weights:  87
## initial  value 7101.790805
## iter  10 value 4204.539886
## iter  20 value 4078.260931
## iter  30 value 4064.607656
## iter  40 value 4019.862165
## iter  50 value 3861.077546
## iter  60 value 3772.640340
## iter  70 value 3731.624402
## iter  80 value 3668.072939
## iter  90 value 3649.279076
## iter 100 value 3644.071019
## final  value 3644.071019
## stopped after 100 iterations
```

```
## # weights:  107
## initial  value 7571.813738
## iter  10 value 4217.877827
## iter  20 value 4143.272971
## iter  30 value 4113.802485
## iter  40 value 4075.546542
## iter  50 value 3980.018813
## iter  60 value 3718.894025
## iter  70 value 3593.044625
## iter  80 value 3541.231287
## iter  90 value 3518.279148
## iter 100 value 3510.587026
## final  value 3510.587026
## stopped after 100 iterations
## # weights:  127
## initial  value 8196.923617
## iter  10 value 4126.436335
## iter  20 value 4026.817717
## iter  30 value 3887.009597
## iter  40 value 3805.152903
## iter  50 value 3745.174365
## iter  60 value 3699.297554
## iter  70 value 3675.390267
## iter  80 value 3648.547586
## iter  90 value 3625.974792
## iter 100 value 3611.142809
## final  value 3611.142809
## stopped after 100 iterations
## # weights:  147
## initial  value 6681.675099
## iter  10 value 4207.476004
## iter  20 value 4105.144017
## iter  30 value 3834.260661
## iter  40 value 3675.354781
## iter  50 value 3595.773847
## iter  60 value 3533.196296
## iter  70 value 3501.527209
## iter  80 value 3492.440875
## iter  90 value 3486.178729
## iter 100 value 3479.750396
## final  value 3479.750396
## stopped after 100 iterations
## # weights:  167
## initial  value 9253.504123
## iter  10 value 4136.575882
## iter  20 value 4029.660111
## iter  30 value 3981.009773
## iter  40 value 3933.160868
## iter  50 value 3830.559118
## iter  60 value 3616.991987
## iter  70 value 3500.847400
## iter  80 value 3455.068800
## iter  90 value 3418.313539
## iter 100 value 3405.785785
```

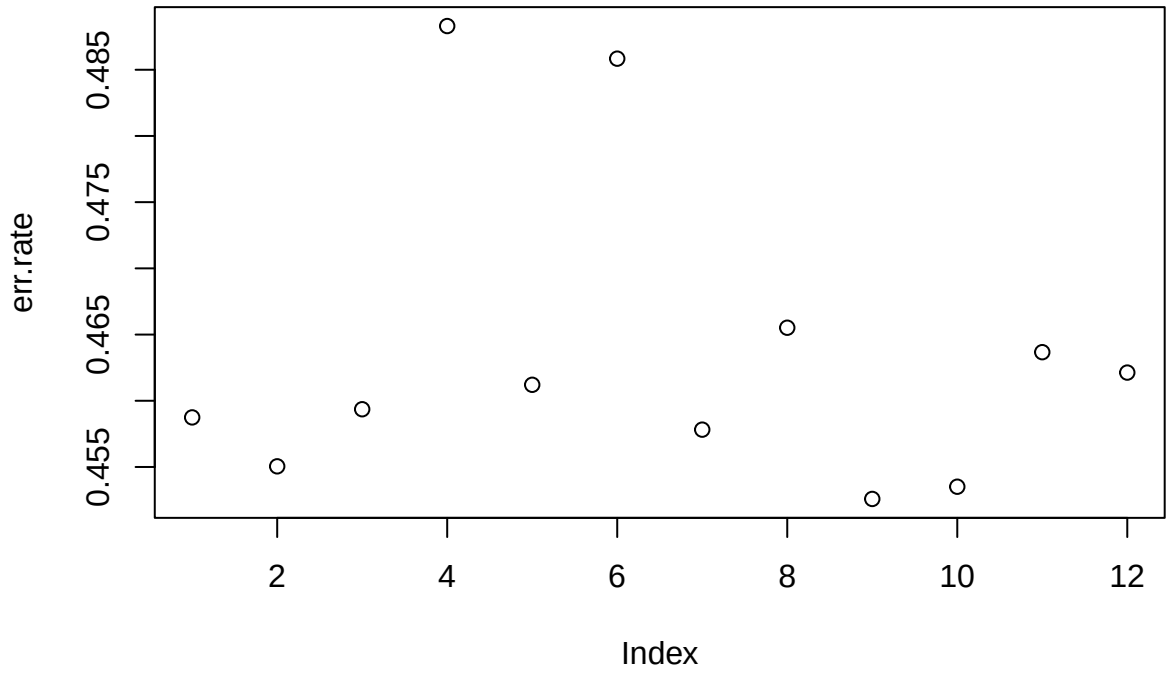
```
## final  value 3405.785785
## stopped after 100 iterations
## # weights:  187
## initial  value 7161.848312
## iter  10 value 4225.295899
## iter  20 value 4164.000369
## iter  30 value 4113.715935
## iter  40 value 3960.059351
## iter  50 value 3782.323093
## iter  60 value 3682.893079
## iter  70 value 3566.664441
## iter  80 value 3534.444564
## iter  90 value 3518.960091
## iter 100 value 3484.804983
## final  value 3484.804983
## stopped after 100 iterations
## # weights:  207
## initial  value 6040.084600
## iter  10 value 4193.819211
## iter  20 value 4124.975346
## iter  30 value 4094.505594
## iter  40 value 4031.836069
## iter  50 value 3900.086895
## iter  60 value 3715.151340
## iter  70 value 3571.334502
## iter  80 value 3519.323584
## iter  90 value 3429.999422
## iter 100 value 3389.789339
## final  value 3389.789339
## stopped after 100 iterations
## # weights:  227
## initial  value 9953.715028
## iter  10 value 4251.952035
## iter  20 value 4143.306805
## iter  30 value 4126.530123
## iter  40 value 4004.410349
## iter  50 value 3926.009488
## iter  60 value 3841.577308
## iter  70 value 3675.099237
## iter  80 value 3541.185587
## iter  90 value 3497.579560
## iter 100 value 3480.180268
## final  value 3480.180268
## stopped after 100 iterations
## # weights:  247
## initial  value 11599.707194
## iter  10 value 4220.772420
## iter  20 value 4123.031903
## iter  30 value 4061.621204
## iter  40 value 4009.389936
## iter  50 value 3711.165221
## iter  60 value 3570.756933
## iter  70 value 3471.889656
## iter  80 value 3398.178730
```

```
## iter  90 value 3362.708132
## iter 100 value 3345.988627
## final  value 3345.988627
## stopped after 100 iterations
```

```
err.rate
```

```
##  [1] 0.4587438 0.4550493 0.4593596 0.4883005 0.4612069 0.4858374 0.4578202
##  [8] 0.4655172 0.4525862 0.4535099 0.4636700 0.4621305
```

```
plot(err.rate)
```




```
# 9 hidden nodes produce the lowest prediction error rate
```

2. Cross Validation with Optimal number of hidden nodes

```
set.seed(1)

ann_wine_quality_optimal <- nnet(quality ~ .,
                         data = wine_train,
                         size = 5)
```

```
## # weights:  107
## initial  value 6969.078871
## iter  10 value 4440.380327
## iter  20 value 4179.151724
## iter  30 value 4112.274669
## iter  40 value 4061.590295
```

```
## iter  50 value 3821.928318
## iter  60 value 3655.338146
## iter  70 value 3560.391221
## iter  80 value 3500.531468
## iter  90 value 3481.873217
## iter 100 value 3475.095482
## final  value 3475.095482
## stopped after 100 iterations
```

```
yhat_ann <- predict(ann_wine_quality_optimal, newdata = wine_test, type = "class")

ann_classification_rate <- mean(wine_test$quality == yhat_ann)

mad_ann <- mean(abs(as.numeric(wine_test$quality) - as.numeric(yhat_ann)))
```

Summary: Which model preformed at predictiong the quality of a wine given it's chariterisitcs?

Using cross validation evaluating the performance of each model, the random forest model with 501 trees and 8 mrty produced the lowest error rate of 15.2%.

```
classification_rates <- data.frame(
  Model = c("K-Nearest Neighbors", "Classifcation Tree", "Random Forest", "Support Vector Machines", "A
  Classification_Rate = c(knn_cv_classification_rate,
                          tree_pruned_classification_rate,
                          forest_classification_rate,
                          svm_cv_classification_rate,
                          ann_classification_rate),
  Error_Rate = c(1- knn_cv_classification_rate,
                       1- tree_pruned_classification_rate,
                       1- forest_classification_rate,
                       1- svm_cv_classification_rate,
                       1- ann_classification_rate),
  Mean_Absolute_Deviation = c(mad_knn,
                              mad_tree,
                              mad_forest,
                              mad_svm,
                              mad_ann)
)

classification_rates %>%
  arrange(desc(Classification_Rate))
```

```
##                       Model Classification_Rate Error_Rate
## 1             Random Forest           0.6253079  0.3746921
## 2 Artifical Nerual Network           0.5412562  0.4587438
## 3      K-Nearest Neighbors           0.5172414  0.4827586
## 4       Classifcation Tree           0.5138547  0.4861453
## 5  Support Vector Machines           0.3682266  0.6317734
##   Mean_Absolute_Deviation
## 1               0.4227217
## 2               1.9048645
```

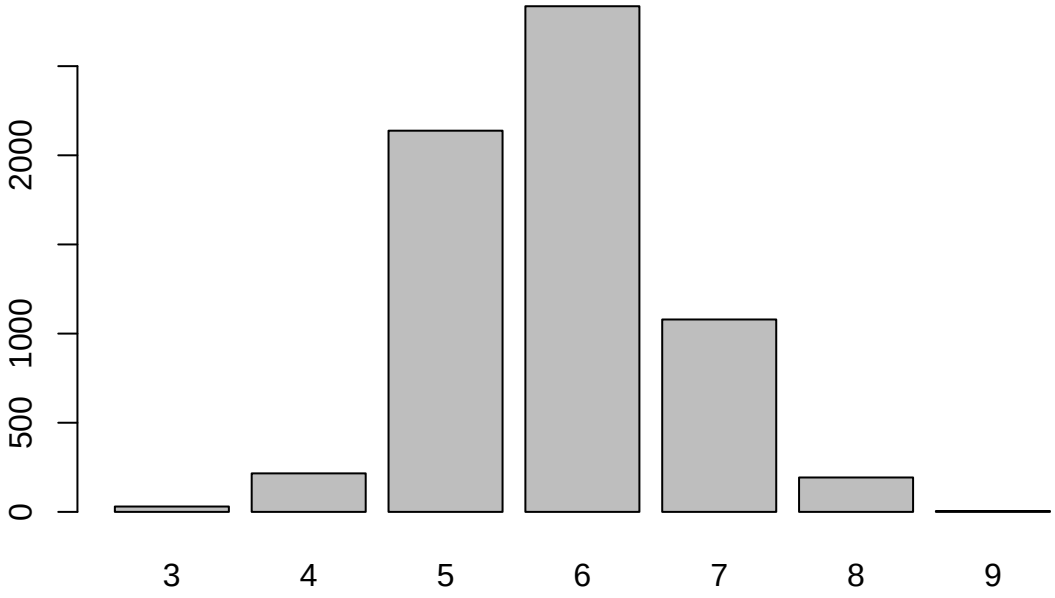
```
## 3               0.6114532
## 4               0.5467980
## 5               0.8035714
```

Research Question #2: What are the characteristics of good Portuguese wine?

Method #6: Clustering - K Means

Quality categorical variable follows a fairly normal distribution; therefore, Converting the variable back into a numeric value to cluster. The clusters with the highest average quality score should have the highest portion of high quality wine. We see the characteristics the wine critics are looking for in a good wine.

```
plot(wine_data$quality)
```



1. K-Means Clustering

```
set.seed(1)

wine_data_numeric <- wine_data %>%
  select(-type)

wine_data_numeric$quality <- as.numeric(wine_data_numeric$quality)

kmeans_result <- kmeans(wine_data_numeric, 6)

kmeans_result
```

```
## K-means clustering with 6 clusters of sizes 987, 1364, 1026, 394, 1108, 1618
##
## Cluster means:
##    quality fixed.acidity volatile.acidity citric.acid residual.sugar  chlorides
```

```
## 1 3.775076      7.300912        0.3807953   0.2975583       3.059777 0.06113779
## 2 3.921554      6.879142        0.2826540   0.3342669       6.721591 0.04860191
## 3 3.634503      6.956335        0.2855799   0.3536257       8.823148 0.05119786
## 4 3.530457      7.015482        0.3073604   0.3574365      10.025000 0.05227411
## 5 3.721119      8.391155        0.5075090   0.2695126       2.476038 0.08248827
## 6 4.011125      6.854141        0.2898640   0.3203090       4.592460 0.04505192
##   free.sulfur.dioxide total.sulfur.dioxide   density       pH sulphates
## 1            21.25836             75.17933 0.9937686 3.234883 0.5502229
## 2            37.15762            143.84457 0.9944210 3.198226 0.4931745
## 3            46.87378            178.50341 0.9958909 3.182992 0.5068421
## 4            55.20812            222.05964 0.9967887 3.177411 0.5179949
## 5            11.01218             27.84206 0.9964324 3.304576 0.6412004
## 6            27.57231            111.31119 0.9930398 3.199178 0.4952596
##     alcohol
## 1 10.870973
## 2 10.397676
## 3  9.846118
## 4  9.555584
## 5 10.590539
## 6 10.909652
##
## Clustering vector:
##    [1] 5 1 1 1 5 5 1 5 5 6 1 6 1 5 2 2 6 1 5 1 1 1 5 1 5 5 5 5 5 5 1 5 6 1 5 5 5
##   [38] 5 5 1 1 5 5 5 5 1 6 5 5 6 5 5 5 6 1 5 5 6 1 5 1 6 5 1 5 5 5 5 1 5 5 6 1 5
##   [75] 1 5 5 5 1 6 5 1 1 5 1 5 2 5 6 5 2 2 2 5 1 6 5 5 5 5 5 5 5 1 5 1 1 1 1 2 5
##  [112] 6 6 5 5 1 5 5 1 1 1 1 5 5 6 1 5 5 5 5 2 6 6 5 5 1 5 5 6 6 1 5 1 5 1 2 6 1
##  [149] 5 5 5 1 6 6 2 6 2 6 5 1 5 5 5 6 6 6 6 5 5 1 5 5 5 5 5 5 5 5 5 5 5 6 5 1 1
##  [186] 1 1 5 2 2 6 5 6 5 5 6 1 5 1 5 5 2 5 5 5 5 5 6 6 5 5 1 5 1 5 6 5 5 5 2 1 1
##  [223] 5 5 1 1 1 5 1 5 6 5 1 5 5 5 5 5 5 5 1 5 6 5 5 5 5 1 5 5 5 5 5 1 5 6 5 1 5
##  [260] 5 5 1 5 5 5 5 1 5 5 5 1 5 1 5 6 1 5 5 5 5 5 5 1 5 1 1 5 1 5 6 5 5 5 5 1 5
##  [297] 1 5 5 5 5 5 5 5 1 5 1 5 5 5 5 6 6 2 1 1 6 1 1 1 1 1 5 1 5 5 5 5 5 5 5 5 6
##  [334] 5 5 5 5 1 6 5 5 5 5 5 5 1 5 5 5 5 5 5 5 1 3 5 1 5 5 5 1 5 5 5 5 5 5 5 1 5
##  [371] 1 5 5 1 5 5 5 5 5 1 5 5 5 5 1 5 5 5 1 5 1 5 5 1 5 5 2 5 5 5 2 5 1 5 5 5 5
##  [408] 5 5 5 1 1 1 5 6 2 5 6 5 1 1 5 1 5 1 5 5 5 5 5 5 1 5 5 1 5 1 5 1 5 5 5 5 5
##  [445] 5 5 1 5 5 5 5 5 5 5 1 5 5 1 5 5 5 5 5 6 5 5 1 5 5 1 5 1 1 5 5 5 5 5 5 5 5
##  [482] 5 5 5 5 5 5 5 1 5 1 5 5 1 1 5 5 6 5 1 5 1 1 5 5 5 5 5 1 5 5 1 5 5 5 2 5 5
##  [519] 5 6 5 5 2 2 6 1 6 1 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 1 5 6 5 5 1 5 5 5 5 6 5
##  [556] 5 5 5 5 5 5 6 6 1 5 5 5 5 5 5 5 5 5 5 1 5 5 6 6 5 5 5 5 5 1 5 5 6 5 5 1 2
##  [593] 1 5 5 6 5 5 5 5 5 5 5 5 1 5 5 1 1 5 1 5 5 5 6 1 1 5 5 5 6 6 5 5 1 1 5 5 5
##  [630] 1 5 5 5 1 6 5 2 2 5 1 5 1 5 1 5 5 5 5 5 2 5 2 1 5 5 1 5 5 5 5 5 5 5 5 1 5
##  [667] 5 5 5 5 5 5 2 5 5 5 5 5 5 1 5 5 5 1 5 2 5 5 5 5 5 5 5 1 1 6 6 5 5 5 1 5 1 5 5
##  [704] 1 5 1 5 5 5 5 1 6 5 5 1 5 5 5 5 5 5 6 5 2 5 5 5 5 5 5 1 5 5 5 1 5 5 5 1 6 5
##  [741] 5 2 5 6 6 5 1 1 5 5 5 5 1 5 5 5 5 5 5 5 1 6 5 5 5 5 5 5 1 6 6 5 6 2 2 5 5 5 5
##  [778] 5 5 1 5 5 6 5 1 1 1 1 1 6 1 2 1 5 5 1 1 5 5 5 6 5 1 5 5 5 5 5 5 5 5 5 5 5
##  [815] 5 5 5 5 5 5 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 1 1 5 1 6 6 5 5 5 5 5 1 1 5 5 5 5 5 5 5
##  [852] 5 6 1 1 5 1 5 5 5 1 1 5 1 1 1 5 5 5 5 5 5 5 5 5 5 5 5 5 1 1 5 5 5 1 1 5 5 1
##  [889] 1 6 1 1 5 1 1 5 5 5 5 5 5 5 5 5 5 5 6 1 5 1 5 5 5 5 5 5 5 5 1 5 1 5 5 1 5 5 5
##  [926] 1 1 1 5 5 5 5 1 5 5 1 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
##  [963] 5 5 5 5 5 6 5 5 5 5 5 5 5 1 1 6 5 5 5 5 5 1 5 5 5 5 1 5 5 5 1 5 1 1 5 5 5 5
## [1000] 5 5 5 5 5 5 5 5 5 5 5 1 5 5 5 5 5 5 5 5 6 6 5 5 5 5 5 5 5 5 5 5 1 1 5 5 5 5 5 5 5
## [1037] 5 1 5 5 5 5 5 5 5 1 5 5 1 5 5 1 5 5 5 5 1 1 5 1 1 5 5 5 5 5 5 5 5 5 5 5 1 5 1 1
## [1074] 5 1 1 5 5 5 4 5 4 1 1 1 1 5 5 5 5 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 1
## [1111] 5 1 5 5 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 1 1 1 5 2 5 5 5 5 5 5 6 1 1 1 5 5 1 1 5
## [1148] 5 5 5 5 1 5 5 1 5 1 6 1 5 5 5 5 5 5 5 5 1 5 5 5 5 5 5 5 1 1 1 5 5 1 5 5 5 1 1
```

```
## [1185] 6 5 5 5 6 5 5 5 5 5 1 1 6 5 1 6 5 5 5 1 5 5 5 1 5 5 5 1 5 5 5 5 1 1 5 1 5
## [1222] 5 1 5 5 1 5 5 1 1 1 6 1 5 5 6 5 5 5 5 1 1 5 1 3 5 5 5 5 5 5 1 5 5 5 5 1 1
## [1259] 5 5 1 5 1 5 1 5 5 5 1 1 1 1 5 5 5 1 5 5 1 5 1 1 5 1 5 5 5 5 6 6 5 1 5 5 1
## [1296] 1 1 5 5 5 5 1 5 1 1 1 1 5 1 1 1 5 5 1 1 1 5 5 1 1 1 5 5 1 5 5 5 5 5 1 1 1
## [1333] 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 1 5 1 1 5 5 5 1 6 5 5 1 5 1 5 5 5 6 1
## [1370] 5 1 5 1 1 5 6 5 1 5 5 5 5 6 6 1 1 5 5 5 6 5 5 5 1 1 5 5 6 5 5 2 2 5 5 5 5
## [1407] 5 1 1 1 5 5 5 1 5 5 5 5 5 2 5 1 1 5 5 5 1 5 1 1 5 1 5 5 6 6 1 5 5 1 5 6 5
## [1444] 5 1 6 5 5 1 5 5 5 1 1 5 5 1 1 5 5 1 5 5 5 1 1 1 5 1 5 5 5 5 5 1 1 1 1 5 5
## [1481] 5 5 5 5 5 5 5 5 5 5 5 5 5 2 5 5 2 5 5 5 5 1 1 5 5 5 5 5 5 5 5 5 5 1 1 1 1
## [1518] 5 5 5 5 5 1 1 5 5 5 5 1 1 5 5 5 1 5 5 5 5 5 1 5 5 5 5 5 5 5 5 5 5 5 5 1 5
## [1555] 5 5 5 5 6 2 2 2 5 5 5 5 1 5 5 5 5 5 6 5 1 5 5 5 5 5 5 5 5 6 5 5 1 5 6 1 5
## [1592] 5 5 5 5 1 5 5 5 3 6 6 3 3 6 2 3 6 6 1 6 1 2 3 6 6 1 3 2 1 6 6 3 2 4 2 2 6
## [1629] 6 2 5 6 6 3 6 2 2 2 2 3 3 2 2 2 3 4 4 2 2 3 6 1 6 2 3 4 2 1 6 6 2 2 1 6 6
## [1666] 6 3 6 6 4 4 3 1 6 6 1 1 6 6 6 3 3 4 2 2 2 4 2 2 2 4 2 2 2 4 2 1 1 2 3 3 3
## [1703] 3 3 6 3 3 3 3 3 4 3 2 2 6 2 1 3 3 1 2 2 2 2 2 6 4 3 3 1 4 4 3 4 3 6 2 6 1
## [1740] 1 2 6 6 1 6 3 6 1 6 2 2 6 6 1 3 3 6 6 6 6 2 1 3 4 3 3 6 2 6 2 6 6 2 3 2 1
## [1777] 3 6 3 3 3 3 4 4 4 3 6 6 4 4 3 6 6 3 3 3 4 4 4 3 4 4 2 6 2 2 6 1 3 1 6 2 6
## [1814] 6 2 3 2 3 3 3 3 6 2 2 2 3 4 3 2 2 3 4 3 3 3 3 4 6 2 3 1 6 4 2 4 2 1 1 6 3
## [1851] 3 2 6 2 2 6 6 6 1 6 2 6 4 2 2 3 2 2 2 3 2 2 6 4 3 3 1 1 6 6 6 3 3 3 3 3 3
## [1888] 3 3 3 2 3 2 3 3 3 3 2 6 1 1 1 3 3 3 3 3 2 2 6 3 2 3 3 2 2 2 2 1 1 2 2 2 3
## [1925] 4 4 2 3 1 6 2 1 2 1 1 6 3 2 2 2 6 2 2 2 2 1 3 6 2 2 2 6 2 4 3 3 3 2 2 3 1
## [1962] 2 3 6 6 2 2 3 1 2 3 3 2 1 1 6 6 2 6 6 3 2 2 6 1 6 4 2 3 3 1 6 1 3 3 6 6 3
## [1999] 1 6 3 6 4 3 3 6 6 6 6 2 2 1 1 2 2 6 4 6 2 2 4 4 4 3 4 4 3 1 3 3 1 4 2 6 6
## [2036] 4 3 4 6 6 2 2 4 2 1 2 6 6 6 3 2 2 6 6 6 1 6 4 4 6 3 3 1 3 6 2 1 3 3 2 3 1
## [2073] 2 6 4 6 6 2 3 3 6 2 3 4 6 6 1 3 3 1 1 3 2 6 3 3 2 3 3 4 3 4 4 2 2 6 3 2 3
## [2110] 2 2 2 1 6 3 2 2 1 5 2 2 6 6 1 2 5 6 6 6 6 2 3 3 3 3 3 3 1 4 3 3 2 2 3 2 3
## [2147] 1 6 3 3 6 6 3 6 6 2 2 3 2 2 2 3 2 3 6 5 2 2 3 3 6 3 2 2 3 3 2 6 2 3 6 3 1
## [2184] 2 1 2 2 6 2 2 6 2 6 6 2 2 6 6 1 2 6 6 6 2 2 2 2 3 1 6 1 2 2 2 2 6 3 3 2 4
## [2221] 3 2 1 6 2 3 4 4 6 2 2 6 3 2 2 2 2 3 3 6 3 3 2 3 3 2 2 3 3 3 3 3 2 2 6 6 2
## [2258] 3 4 1 2 3 1 3 6 3 3 3 4 3 6 2 6 4 3 4 6 6 6 3 3 3 3 3 4 6 3 4 6 2 3 3 3 4
## [2295] 3 6 4 4 4 3 2 1 6 6 5 4 3 2 6 3 6 6 4 3 2 4 2 2 6 3 3 6 1 6 2 6 6 2 3 2 4
## [2332] 5 3 3 6 3 3 2 6 5 1 2 2 2 6 4 3 3 3 3 3 3 4 2 6 3 3 2 2 3 3 4 2 3 2 4 5 6
## [2369] 6 6 2 2 3 6 6 6 3 3 2 1 4 3 2 3 6 6 6 6 6 6 1 2 1 3 2 4 2 6 1 3 4 4 3 6 3
## [2406] 4 4 4 4 4 6 2 3 4 6 1 2 6 2 1 4 6 6 1 3 2 2 1 1 6 2 6 1 6 6 3 3 3 6 6 3 3
## [2443] 6 6 6 2 1 3 6 6 3 6 2 6 6 3 3 2 3 6 3 2 6 2 6 6 2 3 6 2 2 6 5 1 2 6 6 6 6
## [2480] 2 6 6 6 4 6 3 1 3 1 3 6 6 2 6 1 4 1 1 4 2 6 4 4 2 6 1 6 2 4 2 2 2 5 1 5 2
## [2517] 6 6 6 2 2 2 4 2 6 1 3 3 6 1 4 4 4 4 4 1 6 4 3 3 4 1 2 6 6 4 2 1 1 2 2 6 6
## [2554] 2 2 6 1 1 3 3 6 3 6 2 6 3 2 1 1 6 6 2 1 6 6 2 3 2 1 1 2 1 1 2 6 3 3 2 6 2
## [2591] 1 4 1 3 2 5 2 3 3 1 4 4 2 2 1 2 1 3 2 6 3 6 4 3 6 2 6 2 3 6 6 3 3 3 6 2 1
## [2628] 2 4 2 4 4 4 4 4 2 1 1 6 6 2 1 6 4 6 1 6 1 1 6 2 2 1 6 1 6 6 3 6 3 6 3 4 3 2
## [2665] 6 3 6 1 2 3 6 6 3 3 6 3 2 3 4 6 2 2 3 2 3 6 6 3 6 4 3 2 2 3 6 2 2 6 6 2 6
## [2702] 2 6 3 3 6 1 1 2 1 6 3 6 5 6 1 4 1 2 6 6 1 6 2 3 3 6 6 6 2 6 2 6 6 2 3 3 2
## [2739] 1 1 2 2 2 2 4 2 3 6 4 4 4 1 6 3 3 2 2 3 2 2 2 2 3 1 2 1 6 1 2 6 6 6 6 4 3
## [2776] 3 3 3 6 6 6 6 2 2 4 3 6 2 1 6 2 6 2 4 2 2 2 6 1 6 1 3 6 6 3 3 3 2 6 1 3 2
## [2813] 6 1 3 1 4 6 1 4 2 6 3 6 1 6 6 6 6 4 6 6 2 2 3 1 6 2 2 3 3 2 2 2 4 3 1 1 3
## [2850] 3 2 2 3 2 6 2 4 4 4 4 2 2 4 1 3 6 3 2 3 3 6 2 6 3 2 3 2 6 2 2 3 2 2 3 2 2
## [2887] 6 6 1 6 3 1 6 6 3 2 6 3 3 3 3 4 3 4 6 6 2 6 1 2 6 2 3 3 3 3 3 6 4 6 6 2 2
## [2924] 3 6 3 3 1 6 6 6 4 3 6 3 2 4 4 6 6 2 6 3 3 6 3 2 2 1 6 2 3 4 3 2 3 3 6 2 3
## [2961] 3 6 6 2 2 6 6 3 3 6 3 3 3 6 2 6 2 2 6 2 6 1 6 6 2 6 1 2 3 6 1 1 6 6 6 2 3
## [2998] 3 1 3 4 6 6 1 6 1 1 3 6 6 1 2 6 2 1 6 4 6 6 2 1 6 4 6 6 2 2 1 1 6 1 1 6 2
## [3035] 6 4 1 4 4 3 6 6 6 2 2 6 2 2 3 2 2 2 2 4 2 6 2 2 2 6 3 2 2 2 2 3 6 1 2 2 2
## [3072] 5 2 1 2 3 6 2 6 2 6 3 1 6 6 5 3 4 3 6 6 3 3 3 3 1 6 1 2 2 3 6 6 6 2 4 3 3
## [3109] 2 2 6 2 3 2 2 2 3 6 3 3 6 2 3 2 2 2 3 2 2 2 3 4 6 3 6 5 2 6 6 3 1 6 6 5 2
## [3146] 2 2 6 6 2 2 2 2 3 2 3 2 6 1 2 6 6 2 6 6 2 6 3 2 2 3 3 6 6 3 1 6 3 3 6 1 6
```

```
## [3183] 3 4 3 2 4 3 6 3 3 6 1 2 4 2 4 6 1 2 4 1 1 2 6 6 6 4 4 6 6 1 6 2 2 4 2 4 2
## [3220] 1 6 3 6 2 2 3 6 6 6 6 6 6 6 4 6 2 2 4 6 2 2 2 2 2 2 2 2 6 6 3 2 4 6 3 3 2 6
## [3257] 2 4 4 4 3 6 3 3 6 1 2 1 6 2 1 3 2 3 4 1 2 3 6 3 4 4 4 3 4 2 3 4 6 3 4 4 3
## [3294] 2 2 3 3 4 3 6 4 6 2 2 6 2 3 2 3 1 6 3 2 6 2 6 2 3 1 3 3 2 2 6 2 6 4 6 2 6
## [3331] 4 6 2 3 3 6 3 6 1 2 2 2 3 2 4 1 3 6 2 3 2 3 2 6 2 4 4 1 4 3 6 6 6 4 4 4 3
## [3368] 2 6 3 2 3 3 6 2 3 2 2 2 2 3 2 6 2 2 6 2 6 3 3 6 2 3 6 2 6 2 3 2 3 1 3 2 3
## [3405] 6 6 3 6 3 3 6 1 6 6 2 2 6 6 2 2 6 6 4 3 1 3 4 4 3 3 2 3 2 2 4 6 6 4 6 2 6
## [3442] 4 2 6 2 3 2 4 3 3 6 2 6 3 4 3 3 3 4 3 3 6 4 6 2 3 2 2 2 2 3 3 3 2 2 2 1 4
## [3479] 6 4 6 4 4 6 6 6 3 3 6 4 3 2 2 2 4 3 3 1 6 3 6 3 6 4 3 6 3 6 2 3 1 6 2 2 6
## [3516] 3 1 3 3 2 2 6 1 6 6 6 3 3 3 4 4 1 3 1 3 4 3 1 2 4 2 4 3 3 2 3 3 2 2 4 3 2
## [3553] 3 4 3 1 1 4 1 1 6 6 4 3 3 6 3 2 3 6 3 2 2 2 4 4 2 6 3 4 4 4 4 4 3 2 3 4 6
## [3590] 6 4 6 3 3 3 3 3 3 3 2 2 2 3 3 3 3 1 2 6 6 2 2 2 1 1 6 1 2 2 3 2 4 2 3 2 1
## [3627] 3 2 4 2 2 6 3 6 1 2 2 6 1 4 6 6 6 6 1 6 2 2 3 6 2 2 6 6 2 2 2 4 3 4 6 2 6
## [3664] 3 6 2 2 6 6 3 2 3 3 3 6 6 3 6 6 6 6 6 6 2 2 6 2 2 2 3 3 6 2 2 2 6 2 3 2 2
## [3701] 1 4 2 2 6 4 2 3 3 2 2 2 1 3 2 1 6 6 6 2 6 3 2 6 3 3 4 2 6 2 2 2 2 6 4 2 4
## [3738] 6 6 6 6 3 1 2 3 6 2 6 6 6 6 1 1 3 3 1 1 6 1 6 1 6 1 2 2 2 4 4 4 4 4 6 3 4
## [3775] 4 6 6 3 6 2 6 2 2 6 6 1 2 1 6 6 2 2 6 1 2 1 6 3 4 2 2 3 2 3 3 2 2 6 6 2 2
## [3812] 6 6 6 2 6 6 2 2 6 2 2 3 2 3 3 3 2 2 2 2 1 2 1 2 3 2 2 2 3 3 3 3 4 2 2 2 6
## [3849] 3 4 6 3 3 3 2 6 2 4 2 4 2 1 3 2 3 2 2 2 3 6 2 6 1 6 3 4 1 4 2 1 1 4 4 4 4
## [3886] 2 2 4 6 6 2 4 6 6 2 3 4 6 6 6 6 3 6 2 2 2 2 2 6 2 2 2 1 2 2 6 2 3 1 2 3 6
## [3923] 2 3 6 6 6 6 3 4 2 4 6 4 6 4 3 3 1 2 3 6 2 1 4 3 6 2 3 4 4 6 1 1 6 2 1 4 6
## [3960] 6 6 2 3 4 2 4 3 3 2 6 4 1 1 3 2 3 1 4 3 3 2 6 1 6 3 3 3 6 1 2 2 2 2 4 3 4
## [3997] 6 1 1 6 6 4 6 6 3 6 6 4 6 4 4 4 6 3 6 4 4 6 4 6 3 4 6 4 6 3 4 3 4 3 3 3 4
## [4034] 4 2 3 2 3 2 2 2 3 4 3 4 4 2 6 3 2 2 2 2 4 3 2 2 3 2 2 1 1 3 6 3 3 6 1 6 3
## [4071] 3 6 2 1 6 2 1 4 2 6 4 4 4 3 4 6 6 6 3 6 4 3 2 2 2 6 2 2 3 2 3 6 6 3 3 4 2
## [4108] 3 2 4 4 1 2 2 5 3 4 5 4 3 2 2 3 3 2 2 6 6 2 3 3 3 2 6 2 6 2 2 6 3 2 2 6 6
## [4145] 2 3 6 4 3 6 4 6 2 2 2 3 2 2 2 6 2 6 2 2 4 1 2 4 2 2 1 1 3 3 3 4 4 6 6 3 3
## [4182] 3 3 3 3 3 1 3 2 2 2 2 2 6 2 4 6 4 3 2 3 6 6 6 5 2 4 4 5 3 4 6 2 1 2 2 2 2
## [4219] 2 2 2 2 2 1 3 2 2 4 4 6 6 3 3 3 1 3 3 6 1 1 3 1 2 3 6 1 2 2 3 3 2 1 2 4 4
## [4256] 3 2 3 1 2 3 6 1 1 6 2 6 6 3 2 2 6 5 6 6 6 2 2 6 6 2 2 2 2 6 6 3 2 6 6 6 2
## [4293] 2 6 6 3 6 2 6 1 2 2 2 4 4 4 2 3 3 3 2 3 3 3 3 3 2 1 3 1 3 6 2 3 2 1 6 4 1
## [4330] 3 3 2 1 2 4 6 6 2 6 2 2 3 5 5 6 2 2 3 2 3 6 2 1 5 4 4 6 6 2 2 2 2 6 2 6 6
## [4367] 2 6 2 4 2 2 1 2 2 2 6 1 6 6 2 4 4 4 2 1 2 3 3 4 3 3 6 2 1 6 2 6 2 3 3 1 6
## [4404] 1 6 2 2 3 2 6 6 6 2 1 1 6 1 2 2 6 4 2 1 3 2 3 2 2 3 2 3 1 6 6 2 3 2 1 1 1
## [4441] 6 1 1 6 1 1 1 2 4 2 1 2 6 6 2 2 6 2 6 2 6 1 1 6 6 2 6 6 6 6 1 3 6 2 1 2 3
## [4478] 2 1 1 6 6 1 6 1 6 1 5 2 2 2 4 3 1 3 4 4 1 2 6 6 2 1 6 2 3 1 1 1 3 2 6 3 1
## [4515] 1 2 6 1 6 1 3 2 2 6 3 1 2 2 6 2 1 3 2 1 1 1 3 6 6 2 6 3 6 2 2 6 6 3 2 6 6
## [4552] 6 6 1 6 1 6 6 1 6 6 1 6 2 6 6 6 2 1 2 6 2 6 6 3 6 2 2 6 6 6 4 3 5 6 6 6 6
## [4589] 3 3 2 2 6 6 6 6 2 6 6 6 2 2 6 6 2 3 3 3 4 4 6 6 6 1 6 6 2 6 6 5 1 2 1 6 2
## [4626] 6 2 2 2 2 6 1 3 2 1 3 3 6 3 2 3 3 6 2 6 6 3 2 3 4 3 3 6 1 6 1 6 4 1 3 3 6
## [4663] 3 3 6 3 2 6 1 6 1 3 6 2 1 6 4 1 6 1 3 6 6 1 1 6 3 1 1 1 3 2 6 5 5 6 2 2 6
## [4700] 2 1 6 6 6 2 2 3 2 3 6 6 6 6 6 6 1 6 2 5 4 6 6 1 6 2 2 6 6 6 2 3 6 6 6 4 6
## [4737] 2 6 6 4 5 2 6 2 2 2 2 2 2 1 2 4 1 6 2 6 2 6 2 1 6 2 6 3 6 2 6 1 6 2 1 6 6
## [4774] 6 2 1 2 6 6 1 6 2 1 1 6 1 6 6 2 6 2 6 2 1 6 2 2 2 6 2 2 6 2 3 2 6 6 3 2 6
## [4811] 2 2 3 1 1 6 6 6 6 1 1 6 2 2 1 6 3 4 6 2 6 1 1 2 2 2 2 6 6 6 6 6 6 1 6 6 2
## [4848] 2 2 2 3 6 6 4 3 3 3 3 3 3 6 3 6 4 3 6 2 6 4 6 1 6 6 2 6 2 6 4 2 6 6 2 3 6
## [4885] 6 6 6 3 3 1 1 3 1 6 3 3 3 6 6 1 1 1 6 2 6 2 4 6 6 2 2 6 1 4 1 6 6 6 2 6 1
## [4922] 6 6 6 2 3 1 6 6 3 2 2 2 1 3 3 3 5 6 6 6 6 3 3 3 3 2 5 6 2 6 1 1 2 6 2 6 1
## [4959] 1 1 6 2 6 1 1 6 6 3 6 6 6 2 6 2 2 3 3 2 4 6 2 6 2 3 6 2 4 1 1 6 6 1 6 3 3
## [4996] 2 3 3 6 2 6 2 6 1 6 6 3 3 6 2 2 2 3 2 3 4 1 4 3 2 1 6 2 6 2 2 3 6 2 2 2 6
## [5033] 1 1 1 6 6 4 6 6 6 6 6 6 4 2 3 6 2 2 6 6 2 1 6 1 3 2 6 2 6 3 6 6 3 6 6 4 6 2
## [5070] 3 6 2 6 3 3 1 2 3 6 6 2 6 1 1 6 6 3 1 1 2 6 2 2 6 6 2 3 2 6 2 6 3 1 2 6 2
## [5107] 6 3 3 3 1 6 1 6 1 3 6 2 2 4 2 3 4 2 2 6 2 5 2 3 3 1 2 3 3 1 6 2 1 6 6 2 2
## [5144] 2 4 2 4 2 2 6 6 6 6 1 6 6 1 6 5 1 1 3 1 1 1 6 1 6 1 1 5 6 1 6 6 2 6 6 1 2
```

```
## [5181] 6 1 6 1 6 2 6 6 6 6 3 2 2 6 6 6 2 3 4 1 2 2 6 1 3 2 1 2 2 3 1 6 2 2 1 3 6
## [5218] 6 6 3 6 1 6 1 3 6 3 3 3 6 6 6 6 2 6 1 6 1 6 2 2 2 2 2 2 2 1 6 2 6 6 3 6 3 6
## [5255] 2 3 2 2 2 6 2 1 2 2 2 2 6 1 6 2 6 1 1 6 3 1 3 6 6 3 6 6 3 6 3 3 6 3 6 2 6
## [5292] 6 6 6 2 6 3 6 3 2 2 3 3 3 3 1 6 3 2 5 3 3 3 6 3 3 6 2 4 2 2 2 2 6 6 6 2 1
## [5329] 6 3 3 6 6 6 1 1 2 2 2 2 2 2 2 2 2 6 3 2 2 2 6 3 3 2 2 6 3 3 2 6 6 5 1 2 3
## [5366] 3 3 2 3 2 2 2 6 4 6 2 6 6 6 4 6 6 3 2 6 3 4 4 3 3 3 2 3 6 2 6 1 2 3 5 1 2
## [5403] 6 5 1 1 6 6 6 3 2 2 3 2 6 2 2 6 6 2 2 4 3 6 1 6 6 2 6 6 2 2 6 2 6 6 6 2 6
## [5440] 1 3 6 6 1 3 6 3 1 1 6 6 6 6 6 2 6 6 2 4 4 3 3 3 6 6 6 4 3 3 2 3 3 3 3 6 2
## [5477] 6 3 1 6 6 6 2 6 1 2 1 6 2 6 6 2 1 2 1 3 3 6 2 1 5 1 1 6 1 1 1 2 2 2 6 1 6
## [5514] 2 6 2 4 6 6 2 3 1 6 6 6 1 6 3 3 6 6 6 1 1 3 6 2 6 2 2 2 6 6 3 2 6 2 6 2 2
## [5551] 2 1 6 2 1 2 1 6 6 3 3 6 6 3 6 6 3 3 6 2 6 6 3 3 2 6 6 3 3 2 3 2 6 6 6 6 4
## [5588] 2 6 4 6 1 6 6 6 6 2 6 6 6 6 6 6 6 6 5 6 6 2 6 2 6 3 3 6 2 6 3 6 1 6 2 2 2
## [5625] 6 2 3 1 2 6 6 6 6 6 3 3 3 6 6 3 2 2 2 2 2 2 6 6 6 2 6 2 2 2 6 2 6 6 6 1 1
## [5662] 6 2 1 6 3 2 3 2 3 1 3 6 1 6 2 6 6 1 6 6 2 3 6 1 6 6 1 2 3 1 6 6 6 6 2 1 2
## [5699] 6 6 3 3 1 1 6 2 2 6 2 3 1 6 1 6 6 2 2 2 2 6 2 3 1 1 2 2 4 4 1 6 4 4 6 1 2
## [5736] 6 4 1 2 2 2 2 2 2 3 6 2 6 1 3 3 6 3 3 3 3 3 3 3 3 3 3 6 6 1 2 6 6 6 3 6 2
## [5773] 6 3 2 3 6 2 4 6 1 3 1 1 4 1 6 6 6 1 2 6 6 6 1 1 2 2 6 6 1 6 3 2 1 2 6 1 6
## [5810] 2 3 2 1 4 4 4 6 3 4 1 2 6 5 1 3 3 1 3 6 6 1 1 6 6 6 6 1 6 1 2 2 2 1 6 6 6
## [5847] 6 6 2 6 1 6 1 6 6 6 6 6 6 2 6 3 2 6 2 6 6 2 3 3 4 6 6 6 6 6 3 5 2 6 6 6 1
## [5884] 1 1 1 6 1 6 6 3 6 3 6 3 6 6 6 2 3 2 3 6 6 6 2 6 1 6 2 6 6 1 6 6 1 2 6 6 3
## [5921] 2 6 2 6 3 3 3 2 3 3 3 2 2 2 2 2 2 2 6 2 2 2 6 6 6 6 2 2 3 1 2 2 3 3 2 3 3
## [5958] 3 4 2 6 6 2 6 2 6 2 6 6 6 1 6 6 3 3 2 1 6 2 2 2 6 1 3 3 3 6 6 6 3 4 3 2 2
## [5995] 2 3 3 3 2 3 2 3 3 3 3 2 1 3 2 3 1 6 1 6 2 3 6 6 6 6 6 6 6 6 6 6 3 3 3 2 1
## [6032] 1 3 2 3 1 3 2 6 2 6 2 6 6 6 6 6 6 2 2 3 3 6 2 3 3 3 6 6 2 2 6 6 2 2 2 2 6
## [6069] 6 1 1 1 2 2 6 6 3 3 6 2 3 2 6 6 1 1 1 6 6 6 1 5 3 1 3 1 2 6 6 3 3 2 6 2 3
## [6106] 6 1 2 6 2 6 6 6 4 6 1 1 5 3 4 3 5 2 2 3 4 1 6 3 3 3 6 6 6 3 2 1 2 1 6 6 1
## [6143] 1 6 6 6 5 3 1 6 6 6 6 6 6 2 4 6 6 6 1 2 6 6 2 3 2 2 6 1 6 6 6 6 2 1 6 6 2
## [6180] 6 2 2 2 6 2 2 6 2 2 3 4 2 6 6 6 6 1 6 1 6 6 2 6 2 6 6 3 1 5 1 1 2 2 3 2 6
## [6217] 6 2 3 6 6 2 6 1 2 3 6 6 1 2 6 3 3 2 6 3 6 3 3 6 2 6 3 6 2 1 6 4 1 1 6 4 1
## [6254] 2 2 2 2 2 1 1 2 6 6 6 6 4 6 6 6 2 3 3 6 2 2 2 6 2 3 2 5 1 6 3 6 2 2 2 2 2
## [6291] 3 3 2 3 6 1 6 6 3 3 2 2 1 4 6 6 6 6 2 6 6 6 6 6 1 1 2 2 1 1 6 2 3 6 6 6 2
## [6328] 6 1 6 2 2 6 1 1 1 2 6 1 3 2 2 2 6 4 2 6 3 3 2 3 1 6 6 1 6 6 2 2 2 6 1 2 6
## [6365] 6 2 3 3 3 3 3 4 1 6 6 2 6 6 6 2 2 2 2 3 1 6 6 3 6 6 2 2 2 2 2 2 2 6 6 6 2 2
## [6402] 6 6 1 6 2 6 6 2 2 1 6 3 6 6 2 6 1 6 3 6 3 6 2 2 2 1 2 1 2 2 2 6 6 6 1 4 6
## [6439] 5 2 3 6 3 3 1 2 6 3 3 3 3 6 1 1 3 3 6 3 6 6 6 1 6 2 6 6 1 6 1 2 6 3 6 6 6
## [6476] 6 1 1 3 3 6 2 2 3 3 6 1 2 6 6 6 6 1 3 6 6 6
##
## Within cluster sum of squares by cluster:
## [1] 241758.6 410085.5 411456.8 396952.5 215548.1 370440.2
##  (between_SS / total_SS =  91.1 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

2. Summary Statistics

```
custom_summary <- wine_data_numeric %>%
  summarise_all(list(
    Mean = ~ mean(., na.rm = TRUE),
    Median = ~ median(., na.rm = TRUE),
```

```
    SD = ~ sd(., na.rm = TRUE),
    Min = ~ min(., na.rm = TRUE),
    Max = ~ max(., na.rm = TRUE),
    shapirowilks = ~ shapiro.test(sample(., size = 4999))$p.value
  )) %>%
  pivot_longer(cols = everything(),
               names_to = c("Variable", "Statistic"),
               names_sep = "_") %>%
  pivot_wider(names_from = Statistic, values_from = value) %>%
 write_csv( "./Data/summary.csv")
```

3. Results of the T-test on the sample means of the lowest and highest clusters accross all varaibles.

```
wine_data_numeric$cluster <- as.factor(kmeans_result$cluster)

wine_data_numeric %>%
  filter(cluster %in% c(4, 6)) %>%
  select(-cluster) %>%
  summarise(across(everything(),
                   ~ {
                     test_result <- t.test(. ~ wine_data_numeric$cluster[wine_data_numeric$cluster %in%
                     c(t_stat = test_result$statistic,
                       p_value = test_result$p.value)
                   },
                   .names = "{col}_t_test")) %>%
  pivot_longer(everything(), names_to = "Variable",
               values_to = "Test_Result") %>%
  mutate(Statistic = rep(c("T Statistic", "P Value"),
                         each =  ncol(wine_data_numeric) - 1)) %>%
  pivot_wider(names_from = Statistic , values_from = Test_Result) %>%
  separate(Variable, into = c("Variable", "Type"), sep = "_") %>%
  mutate(`High Quality Wine has ...` = case_when(
    `T Statistic` > 0 ~ "Lower Level",
    `T Statistic` <= 0 ~ "Higher Level"))
```

```
## Warning: Returning more (or less) than 1 row per ‘summarise()‘ group was deprecated in
## dplyr 1.1.0.
## i Please use ‘reframe()‘ instead.
## i When switching from ‘summarise()‘ to ‘reframe()‘, remember that ‘reframe()‘
##   always returns an ungrouped data frame and adjust accordingly.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

```
## Warning: Expected 2 pieces. Additional pieces discarded in 12 rows [1, 2, 3, 4, 5, 6, 7,
## 8, 9, 10, 11, 12].
```

```
## # A tibble: 12 x 5
##    Variable             Type  ‘T Statistic‘ ‘P Value‘ High Quality Wine has ..~1
##    <chr>                <chr>         <dbl>     <dbl> <chr>
```

```
##  1 quality              t            -10.8  2.70e- 25 Higher Level
##  2 fixed.acidity        t              3.71 2.20e-  4 Lower Level
##  3 volatile.acidity     t              2.96 3.20e-  3 Lower Level
##  4 citric.acid          t              4.98 8.34e-  7 Lower Level
##  5 residual.sugar       t             22.9  2.08e- 83 Lower Level
##  6 chlorides            t              4.59 5.54e-  6 Lower Level
##  7 free.sulfur.dioxide  t             24.9  7.22e- 86 Lower Level
##  8 total.sulfur.dioxide t             93.3  1.16e-287 Lower Level
##  9 density              t             31.0  2.30e-137 Lower Level
## 10 pH                   t             -2.86 4.41e-  3 Higher Level
## 11 sulphates            t              4.10 4.62e-  5 Lower Level
## 12 alcohol              t            -29.4  8.07e-141 Higher Level
## # i abbreviated name: 1: ‘High Quality Wine has ...‘
```